

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 4 JDBC — Conexión a bases desde Java

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Ejercicio 1: Alta y listado de clientes (JDBC)	2
3. Ejercicio 2: Invocar la función de máxima cotización	5
4. Ejercicio 3: Listado de metadatos (PK, UQ, FK)	7
5. Decisiones transversales	12
Conclusión	12

1. Introducción

El Práctico 4 cierra el bloque de fundamentos del cuatrimestre moviéndose desde SQL puro hacia el *cliente*: cómo se accede a las bases que armamos en prácticos anteriores desde una aplicación Java mediante la API **JDBC** (*Java DataBase Connectivity*). Los tres ejercicios cubren los tres usos canónicos de JDBC:

- **Ejercicio 1:** `PreparedStatement` para alta y `ResultSet` para listado, sobre la base de facturación del Práctico 1.a (MySQL y PostgreSQL).
- **Ejercicio 2:** `CallableStatement` para invocar la función `fn_actualizar_cotizacion_maxima` creada en el Práctico 3, sobre PostgreSQL.
- **Ejercicio 3:** `DatabaseMetaData` para listar tablas, columnas, claves primarias, únicas y foráneas de la base del Práctico 1.c, en PostgreSQL y Oracle.

Toda la solución está organizada como tres aplicaciones de consola independientes, sin frameworks de persistencia (ni Hibernate ni JPA): JDBC “pelado” como pide el enunciado.

Convenciones de proyecto. Cada ejercicio vive en su propia carpeta (`ejercicio-01`, `ejercicio-02`, `ejercicio-03`) con la misma estructura: `src/` (código), `lib/` (drivers JDBC), `config/` (`*.properties`) y un `README.md` con las instrucciones de compilación y ejecución. Los datos sensibles (host, usuario, clave) viven en los `.properties`, no en el código.

Verificación de las resoluciones existentes

Antes de redactar este documento se revisaron las tres resoluciones. Conclusiones:

- Todas usan `try-with-resources` para cerrar `Connection`, `PreparedStatement` y `ResultSet`.
- Las consultas son siempre parametrizadas (`PreparedStatement/CallableStatement`); no hay concatenación insegura.
- Cada aplicación carga el driver con `Class.forName(...)` y establece la conexión con `DriverManager.getConnection(...)`.
- La configuración se externaliza en archivos `.properties`, lo que permite cambiar de motor sin recompilar.
- En el Ejercicio 2 se deja un *fallback* con `SELECT * FROM funcion(...)` para los entornos en los que el escape `{ ? = call ... }` no funciona en PostgreSQL.
- En el Ejercicio 3 se separa la carga de tablas del armado de la información por tabla, y se descartan los índices que coinciden con la PK al listar las claves únicas.

No se encontraron errores. Lo que sigue documenta cada solución explicando las decisiones de diseño y los métodos JDBC involucrados.

2. Ejercicio 1: Alta y listado de clientes (JDBC)

Enunciado. Sobre la base del ejercicio 1.a del Práctico 1 (facturación), implementar en Java por consola dos operaciones: *alta* de un cliente y *listado* de todos los clientes cargados. Hay que resolverlo sobre los motores donde se creó la base (MySQL y PostgreSQL).

Estructura de la solución

Tres clases en el paquete `ar.edu.unrc.bd2.practico4.ej1`:

- **Cliente:** *value object* inmutable con los cinco campos del cliente.

- **ClienteDao**: encapsula el acceso a JDBC. Recibe la `Connection` desde afuera (no la administra) y expone dos métodos: `insertar` y `listar`.
- **CientesCliApp**: el `main`, que carga la configuración, abre la conexión y ofrece un menú de consola.
- **DbConfig**: lee el `.properties` y expone el nombre completo de la tabla (`practico1a.cliente` o `practico1a_mysql.cliente`).

Configuración por motor (.properties)

```
# config/db.postgres.properties
db.driver=org.postgresql.Driver
db.url=jdbc:postgresql://localhost:5432/postgres
db.user=postgres
db.password=root
db.table.cliente=practico1a.cliente
```

```
# config/db.mysql.properties
db.driver=com.mysql.cj.jdbc.Driver
db.url=jdbc:mysql://localhost:3306/practico1a_mysql
db.user=root
db.password=root
db.table.cliente=cliente
```

Apertura de la conexión

El método `createConnection()` de `DbConfig` sigue el patrón clásico:

```
public Connection createConnection() throws SQLException, ClassNotFoundException {
    Class.forName(driverClass);
    return DriverManager.getConnection(url, user, password);
}
```

Por qué `Class.forName`. Aunque desde JDBC 4.x los drivers se registran automáticamente vía `ServiceLoader`, forzar la carga con `Class.forName` es la forma robusta y explícita que muestra el teórico, y evita problemas si el driver no está empaquetado con `META-INF/services`.

Acceso a datos: `ClienteDao`

```
public int insertar(Connection connection, Cliente cliente) throws SQLException {
    String sql = "INSERT INTO " + clienteTable
        + " (nro_cliente, apellido, nombre, direccion, telefono) VALUES (?, ?, ?, ?, ?)";

    try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
        preparedStatement.setInt(1, cliente.getNroCliente());
        preparedStatement.setString(2, cliente.getApellido());
        preparedStatement.setString(3, cliente.getNombre());
        preparedStatement.setString(4, cliente.getDireccion());
        preparedStatement.setString(5, cliente.getTelefono());
        return preparedStatement.executeUpdate();
    }
}

public List<Cliente> listar(Connection connection) throws SQLException {
    String sql = "SELECT nro_cliente, apellido, nombre, direccion, telefono FROM "
        + clienteTable + " ORDER BY nro_cliente";

    List<Cliente> clientes = new ArrayList<>();
```

```

try (PreparedStatement preparedStatement = connection.prepareStatement(sql);
    ResultSet resultSet = preparedStatement.executeQuery()) {

    while (resultSet.next()) {
        clientes.add(new Cliente(
            resultSet.getInt("nro_cliente"),
            resultSet.getString("apellido"),
            resultSet.getString("nombre"),
            resultSet.getString("direccion"),
            resultSet.getString("telefono")
        ));
    }
}
return clientes;
}

```

Lectura del código.

- Las dos sentencias se construyen con ? como placeholders. Después se asignan los valores con `setInt/setString` sobre el `PreparedStatement`. Esto es lo que protege contra *SQL Injection*: el motor recibe la sentencia y los valores por separado.
- `executeUpdate` se usa para `INSERT`: devuelve la cantidad de filas afectadas. Se espera 1 para un alta correcta.
- `executeQuery` se usa para `SELECT`: devuelve un `ResultSet`.
- El `ResultSet` se recorre con `while (rs.next()). next()` avanza el cursor a la siguiente fila y devuelve `false` cuando no hay más.
- Se usan `getInt(...)` y `getString(...)` *por nombre de columna*. Es menos eficiente que por índice pero deja el código más legible y robusto frente a cambios en el orden de las columnas.
- Todo se rodea con `try-with-resources` para que el `Statement` y el `ResultSet` se cierren incluso si salta una excepción.

Menú de consola

```

private static void ejecutarMenu(Scanner scanner, Connection connection,
                                ClienteDao clienteDao) {

    boolean salir = false;
    while (!salir) {
        mostrarMenu();
        String opcion = scanner.nextLine().trim();
        switch (opcion) {
            case "1": altaCliente(scanner, connection, clienteDao); break;
            case "2": listarClientes(connection, clienteDao); break;
            case "0": salir = true; break;
            default: System.out.println("Opcion invalida.");
        }
    }
}

```

Manejo de errores. El alta atrapa `NumberFormatException` (cuando `nro_cliente` no es entero) y `SQLException` (cuando, por ejemplo, se viola la `UNIQUE` sobre `telefono` del Práctico 1.a). El listado simplemente reporta el error SQL si llega a ocurrir.

Compilación y ejecución

```
# Desde practico-04/ejercicio-01
mkdir -p out
javac -d out src/main/java/ar/edu/unrc/bd2/practico4/ej1/*.java

# PostgreSQL
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej1.ClientesCliApp \
    config/db.postgres.properties

# MySQL
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej1.ClientesCliApp \
    config/db.mysql.properties
```

3. Ejercicio 2: Invocar la función de máxima cotización

Enunciado. Programa Java que invoque la función `fn_actualizar_cotizacion_maxima` del Práctico 3, ejercicio 2, mostrando los parámetros de salida.

Estructura de la solución

- `CotizacionMaximaApp`: el main; lee el `cod_banco` (de argumento o por consola), invoca el `service`, imprime el resultado y consulta la fila actualizada.
- `CotizacionMaximaService`: encapsula la invocación JDBC. Tiene dos caminos: `CallableStatement` (preferido) y `SELECT * FROM funcion(?)` (*fallback*).
- `DbConfig`: lee `db.driver`, `db.url`, `db.user`, `db.password`, `db.function.call`, `db.function.select` y `db.query.banco`.

Configuración (`db.postgres.properties`)

```
db.driver=org.postgresql.Driver
db.url=jdbc:postgresql://127.0.0.1:5432/postgres
db.user=postgres
db.password=

# Invocacion principal por CallableStatement (parametro OUT en posicion 1).
db.function.call={ ? = call practico3_ej2.fn_actualizar_cotizacion_maxima(?) }

# Fallback por SELECT (compatible con cualquier driver/PostgreSQL).
db.function.select=SELECT * FROM practico3_ej2.fn_actualizar_cotizacion_maxima(?)

# Verificacion posterior de la fila actualizada.
db.query.banco=SELECT cod_banco, nombre, cotizacion_maxima FROM practico3_ej2.banco WHERE
    cod_banco = ?
```

CallableStatement en PostgreSQL

La sintaxis estándar JDBC para llamar a un *stored procedure* con un parámetro OUT es el *escape call*: `{ ? = call func(?) }`. JDBC numera los parámetros desde 1; si la función tiene un OUT, se registra primero con `registerOutParameter`.

```
private BigDecimal invocarConCallableStatement(int codBanco, Connection connection)
    throws SQLException {
    try (CallableStatement callableStatement =
        connection.prepareCall(dbConfig.getFunctionCall())) {
        callableStatement.registerOutParameter(1, Types.NUMERIC);
```

```

        callableStatement.setInt(2, codBanco);
        callableStatement.execute();
        return callableStatement.getBigDecimal(1);
    }
}

```

Detalles.

- `prepareCall` es el método de `Connection` para crear un `CallableStatement`.
- `registerOutParameter(1, Types.NUMERIC)` le dice al driver que el primer placeholder es de salida y de tipo `NUMERIC` (el tipo SQL de `cotizacion_maxima`). La constante viene de `java.sql.Types`.
- `setInt(2, codBanco)` coloca el segundo parámetro (entrada `p_cod_banco`).
- `execute()` corre la llamada.
- `getBigDecimal(1)` lee el valor de salida en el placeholder 1 como `BigDecimal` (la correspondencia natural de `NUMERIC` en Java; ver Tabla de tipos del Teórico 5).

Fallback con SELECT

Algunos drivers/versiones de PostgreSQL no aceptan el escape call estándar para funciones que devuelven valor escalar. Para esos casos está el camino alternativo:

```

private BigDecimal invocarConSelect(int codBanco, Connection connection)
    throws SQLException {
    try (PreparedStatement preparedStatement =
        connection.prepareStatement(dbConfig.getFunctionSelect())) {
        preparedStatement.setInt(1, codBanco);
        try (ResultSet resultSet = preparedStatement.executeQuery()) {
            if (!resultSet.next()) {
                throw new SQLException("La funcion no retorno filas de salida.");
            }
            return resultSet.getBigDecimal(1);
        }
    }
}

```

El *service* prueba primero `CallableStatement` y, si algo falla, cae al `SELECT`:

```

public BigDecimal invocarFuncion(int codBanco, Connection connection) throws SQLException {
    try {
        return invocarConCallableStatement(codBanco, connection);
    } catch (SQLException e) {
        return invocarConSelect(codBanco, connection);
    }
}

```

Verificación posterior

Después de invocar la función, el programa lee la fila de `banco` para confirmar que la columna `cotizacion_maxima` quedó con el mismo valor devuelto:

```

public BancoCotizacion consultarBanco(int codBanco, Connection connection) throws
    SQLException {
    try (PreparedStatement preparedStatement =
        connection.prepareStatement(dbConfig.getBancoQuery())) {
        preparedStatement.setInt(1, codBanco);
    }
}

```

```

        try (ResultSet resultSet = preparedStatement.executeQuery()) {
            if (!resultSet.next()) return null;
            return new BancoCotizacion(
                resultSet.getInt("cod_banco"),
                resultSet.getString("nombre"),
                resultSet.getBigDecimal("cotizacion_maxima")
            );
        }
    }
}

```

Salida esperada

```

Funcion invocada: practico3_ej2.fn_actualizar_cotizacion_maxima
Parametro IN   p_cod_banco      = 1
Parametro OUT  p_cotizacion_maxima = 1245.7500
Banco actualizado -> cod_banco=1, nombre=Banco Rio, cotizacion_maxima=1245.7500

```

Compilación y ejecución

```

# Desde practico-04/ejercicio-02
mkdir -p out
javac -d out src/main/java/ar/edu/unrc/bd2/practico4/ej2/*.java

# Interactivo: pide cod_banco por consola.
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej2.CotizacionMaximaApp \
    config/db.postgres.properties

# O con cod_banco como segundo argumento.
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej2.CotizacionMaximaApp \
    config/db.postgres.properties 1

```

4. Ejercicio 3: Listado de metadatos (PK, UQ, FK)

Enunciado. Programa Java que liste, para la base del Práctico 1.c (clientes, automóviles, accidentes), las tablas, sus columnas, claves primarias, claves únicas y claves foráneas. Debe funcionar sobre los motores en los que se creó la base (PostgreSQL y Oracle).

Estructura de la solución

- **MetadataExplorerApp:** main mínimo. Abre la conexión, obtiene el `DatabaseMetaData` y delega en el `reporter`.
- **MetadataReporter:** hace todo el trabajo de extracción y formateo, dividido en cinco pasos (cargar tablas, columnas, PK, UQ, FK).
- **DbConfig:** filtros para `DatabaseMetaData.getTables` (catalog, schemaPattern, tableNamePattern, tableTypes).

Configuración por motor

```

# config/db.postgres.properties
db.driver=org.postgresql.Driver
db.url=jdbc:postgresql://127.0.0.1:5432/postgres
db.user=postgres

```

```
db.password=

db.catalog=
db.schemaPattern=practico1c
db.tableNamePattern=%
db.table.types=TABLE
```

```
# config/db.oracle.properties
db.driver=oracle.jdbc.OracleDriver
db.url=jdbc:oracle:thin:@//localhost:1521/XE
db.user=PRACTIC01C
db.password=PRACTIC01C

# En Oracle, el "schema" es el usuario; debe ir en MAYUSCULAS.
db.catalog=
db.schemaPattern=PRACTIC01C
db.tableNamePattern=%
db.table.types=TABLE
```

Por qué los filtros distintos por motor. En PostgreSQL se usa el esquema `practico1c` dentro de la base `postgres`. En Oracle no hay “schema” separado del usuario: el esquema es el dueño y los nombres viajan en mayúsculas (`PRACTIC01C`). Por eso cada `.properties` usa el valor correspondiente.

Carga de tablas

```
private List<TableRef> loadTables(DatabaseMetaData metaData, DbConfig config)
    throws SQLException {
    List<TableRef> tables = new ArrayList<>();
    try (ResultSet rs = metaData.getTables(
        config.getCatalog(),
        config.getSchemaPattern(),
        config.getTableNamePattern(),
        config.getTableTypes())) {
        while (rs.next()) {
            tables.add(new TableRef(
                rs.getString("TABLE_CAT"),
                rs.getString("TABLE_SCHEM"),
                rs.getString("TABLE_NAME")
            ));
        }
    }
    tables.sort(Comparator
        .comparing((TableRef t) -> nullSafe(t.schema))
        .thenComparing(t -> nullSafe(t.name)));
    return tables;
}
```

Métodos JDBC usados.

- `getTables(catalog, schemaPattern, tableNamePattern, types)`: devuelve las tablas filtradas. Las columnas `TABLE_CAT`, `TABLE_SCHEM`, `TABLE_NAME` y `TABLE_TYPE` están definidas por el estándar JDBC, así que el código no depende del motor.
- Se ordena por esquema y nombre para imprimir el reporte estable.

Columnas

```
private void printColumns(DatabaseMetaData metaData, TableRef table) throws SQLException {
    try (ResultSet rs = metaData.getColumns(table.catalog, table.schema, table.name, "%")) {
        while (rs.next()) {
            String columnName = rs.getString("COLUMN_NAME");
            String typeName = rs.getString("TYPE_NAME");
            int dataType = rs.getInt("DATA_TYPE");
            int size = rs.getInt("COLUMN_SIZE");
            int decimalDigits = rs.getInt("DECIMAL_DIGITS");
            String nullable = rs.getString("IS_NULLABLE");

            String typeDescription = buildTypeDescription(typeName, dataType, size,
decimalDigits);
            String nullDescription = "YES".equalsIgnoreCase(nullable) ? "NULL" : "NOT NULL";

            System.out.println("- " + columnName + " : " + typeDescription
                + " (" + nullDescription + ")");
        }
    }
}
```

Detalles.

- `getColumns(catalog, schema, table, columnPattern)`.
- `DATA_TYPE` es un código de `java.sql.Types`; `TYPE_NAME` el nombre nativo del motor (`VARCHAR2`, `NUMBER`, `NUMERIC`...).
- `IS_NULLABLE` es "YES" o "NO".
- `buildTypeDescription` reconstruye la representación legible: `VARCHAR(120)` o `NUMBER(10,2)` según el tipo. Sólo agrega tamaños cuando tienen sentido (cadenas y numéricos exactos).

Clave primaria

```
private void printPrimaryKey(DatabaseMetaData metaData, TableRef table) throws SQLException {
    Map<Short, String> pkColumns = new TreeMap<>();
    String pkName = null;

    try (ResultSet rs = metaData.getPrimaryKeys(table.catalog, table.schema, table.name)) {
        while (rs.next()) {
            pkName = rs.getString("PK_NAME");
            short keySeq = rs.getShort("KEY_SEQ");
            pkColumns.put(keySeq, rs.getString("COLUMN_NAME"));
        }
    }

    if (pkColumns.isEmpty()) {
        System.out.println("PK:\n- (sin clave primaria);
        return;
    }

    System.out.println("PK:\n- " + (pkName != null ? pkName : "(sin nombre)")
        + " " + new ArrayList<>(pkColumns.values()));
}
```

KEY_SEQ. La PK puede ser compuesta. Cada fila del `ResultSet` representa una columna y trae `KEY_SEQ` con el orden dentro de la PK. Por eso se usa `TreeMap<Short, String>`: ordena automáticamente por esa clave.

Claves únicas

JDBC no tiene un `getUniqueKeys`. Hay que pedir todos los índices y filtrar los *no únicos* y los que coinciden con la PK (la PK ya genera un índice único pero no es una “clave única adicional”):

```
try (ResultSet rs = metaData.getIndexInfo(table.catalog, table.schema, table.name,
    /* unique = */ true, /* approximate = */ false)) {
    while (rs.next()) {
        short type = rs.getShort("TYPE");
        if (type == DatabaseMetaData.tableIndexStatistic) continue;

        String indexName = rs.getString("INDEX_NAME");
        String columnName = rs.getString("COLUMN_NAME");
        short ordinalPosition = rs.getShort("ORDINAL_POSITION");
        // ...
        uniqueIndexes.computeIfAbsent(indexName, k -> new TreeMap<>())
            .put(ordinalPosition, columnName);
    }
}
```

Después se descartan los índices únicos que tengan *exactamente* las mismas columnas que la PK. Si el resultado queda vacío, se imprime “(sin claves unicas adicionales)”.

Claves foráneas

```
try (ResultSet rs = metaData.getImportedKeys(table.catalog, table.schema, table.name)) {
    while (rs.next()) {
        String fkName = rs.getString("FK_NAME");
        short keySeq = rs.getShort("KEY_SEQ");
        FkPart part = new FkPart(
            rs.getString("FKCOLUMN_NAME"),
            rs.getString("PKTABLE_SCHEM"),
            rs.getString("PKTABLE_NAME"),
            rs.getString("PKCOLUMN_NAME")
        );
        groupedFks.computeIfAbsent(fkName, k -> new TreeMap<>()).put(keySeq, part);
    }
}
```

Detalles.

- `getImportedKeys` devuelve las FK *salientes* de la tabla (las que esta tabla declara hacia otras). `getExportedKeys` sería el camino inverso.
- Una FK puede ser compuesta; KEY_SEQ ordena las columnas. Para imprimirla se reconstruye una línea del tipo `[fkCol1, fkCol2] -> esquema.tabla [pkCol1, pkCol2]`.

Punto de entrada

```
public static void main(String[] args) {
    String configPath = args.length > 0 ? args[0] : "config/db.postgres.properties";
    DbConfig config;
    try {
        config = DbConfig.fromFile(configPath);
    } catch (IOException | IllegalArgumentException e) {
        System.err.println("Error cargando config: " + e.getMessage());
        System.exit(1); return;
    }

    try (Connection connection = config.openConnection()) {
```

```

        DatabaseMetaData metaData = connection.getMetaData();
        System.out.println("Conexion OK -> "
            + metaData.getDatabaseProductName() + " "
            + metaData.getDatabaseProductVersion());

        new MetadataReporter().printMetadata(metaData, config);

    } catch (ClassNotFoundException e) { /* ... */ }
    catch (SQLException e) { /* ... */ }
}

```

Compilación y ejecución

```

# Desde practico-04/ejercicio-03
mkdir -p out
javac -d out src/main/java/ar/edu/unrc/bd2/practico4/ej3/*.java

# PostgreSQL
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej3.MetadataExplorerApp \
    config/db.postgres.properties

# Oracle
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej3.MetadataExplorerApp \
    config/db.oracle.properties

```

Ejemplo de salida

```

Conexion OK -> PostgreSQL 14.10
=====
Tabla: practico1c.cliente
=====
Columnas:
- dni : INT4 (NOT NULL)
- nombre : VARCHAR(60) (NOT NULL)
- apellido : VARCHAR(60) (NOT NULL)
- direccion : VARCHAR(120) (NULL)
- tarifa : INT4 (NOT NULL)
PK:
- pk_cliente [dni]
Claves unicas:
- (sin claves unicas adicionales)
Claves foraneas:
- (sin claves foraneas)

=====
Tabla: practico1c.accidente
=====
...
Claves foraneas:
- fk_acc_cliente [dni] -> practico1c.cliente [dni]
- fk_acc_taller [nro_taller] -> practico1c.taller [nro_taller]
- fk_acc_automovil [patente] -> practico1c.automovil [patente]

```

5. Decisiones transversales

- **Sin frameworks ORM.** Se usan únicamente clases del paquete `java.sql` (`Connection`, `PreparedStatement`, `CallableStatement`, `ResultSet`, `DatabaseMetaData`). Nada de Hibernate, JPA, Spring Data ni similares: el enunciado pide JDBC.
- **Recursos cerrados con try-with-resources.** El compilador genera el `finally` con `close()`, incluso si el cuerpo lanza excepción. Es el patrón seguro y preferido desde Java 7.
- **Configuración externa.** `driver`, `url`, `user`, `password` y los nombres de tabla/función viven en `.properties`. El binario es independiente del entorno.
- **Independencia de motor donde tiene sentido.** El Ejercicio 1 cambia entre MySQL y PostgreSQL sólo cambiando el `.properties`; el Ejercicio 3, entre PostgreSQL y Oracle.
- **Mensajes de error útiles.** En todos los casos se diferencian tres errores típicos: `ClassNotFoundException` (driver ausente), `IOException` (archivo `.properties` mal o ausente) y `SQLException` (problema con la base o la sentencia).

Conclusión

Los tres ejercicios cubren los tres patrones canónicos de uso de JDBC: *ABM con `PreparedStatement`*, *invocación de stored procedure con `CallableStatement`* y *exploración de metadatos con `DatabaseMetaData`*. La estructura repetida en los tres (`App` + `Service/DAO` + `DbConfig` + `.properties`) simplifica leer y comparar las soluciones y deja claro cómo cambia la API según el caso de uso.